Ders 7 Genişletilmiş Notu: Seminer Sunumları ve Huffman Proje Gösterimleri: Çok Katmanlı Görüntü Temsili, BWT-MTF-Aritmetik Kodlama, Transformer ve LSTM ile Kayıpsız Sıkıştırma, Sismik ve Röntgen Verisi Sıkıştırma

Ders 7 Genişletilmiş Notu: Seminer Sunumları ve Huffman Proje Gösterimleri: Çok Katmanlı Görüntü Temsili, BWT-MTF-Aritmetik Kodlama, Transformer ve LSTM ile Kayıpsız Sıkıştırma, Sismik ve Röntgen Verisi Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Düzeni
- 2. Çok Katmanlı Görüntü Temsili
- 3. Mobil Uygulamalarda Metin Sıkıştırma: Aritmetik Kodlama, MTF ve BWT
- 4. Transformer ile Kayıpsız Veri Sıkıştırma
- 5. Gürültü Giderici Otomatik Kodlayıcıyla Denetimsiz Cümle Sıkıştırma
- 6. Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma: LSTM ve Transformer
- 7. DNN ile Sismik Veri Sıkıştırma
- 8. Evrişimsel RNN ile Göğüs Röntgeni Sıkıştırma
- 9. Burrows-Wheeler Dönüşümü
- 10. Proje Gösterimleri: Huffman ile Görüntü Sıkıştırma
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Düzeni

Bu ders de önceki ders gibi öğrenci seminerlerine ayrıldı. Her öğrenci veri sıkıştırmayla ilgili bir makaleyi ya da konferans sunumunu yaklaşık 15 dakikada anlattı. Hoca bazı sunumların ortasında kavramları sordu, sunum sonunda sınıfa soru olup olmadığını sordu ve kendi sorularını yöneltti. Sekiz sunumdan sonra iki öğrenci dönem projelerini (Huffman kodlamasıyla görüntü sıkıştırma) canlı olarak gösterdi.

Sunanlar aşağıda “bir öğrenci” diye anılmıştır. Sunumların bir kısmı klasik dönüşüm ve entropi kodlama fikirleri üzerine kuruludur (çok katmanlı dalgacık temsili, BWT, MTF, aritmetik kodlama); bir kısmı da sinir ağlarının tahminci olarak kullanıldığı sıkıştırma yöntemleridir (Transformer, LSTM, DNN, evrişimsel RNN).

2. Çok Katmanlı Görüntü Temsili

2.1 Problem ve fikir

İlk sunumu yapan öğrenci, çok katmanlı görüntü temsili (multi-layered image representation) makalesini anlattı; öğrenciye göre makale biraz eskidir. Makale, hem pürüzsüz (smooth) hem dokulu (texture) bölgeler içeren görüntüleri verimli kodlama sorununu ele alır. Çözüm olarak görüntü, tutarlı katmanların üst üste binmesi (süperpozisyon) olarak temsil edilir: pürüzsüz bölgeler katmanı, dokular katmanı gibi. Bu yeni yaklaşım hem uzamsal çözünürlük hem de yeniden yapılandırma kalitesi açısından ölçeklenebilirdir.

2.2 Algoritma: ardışık artık sıkıştırma

Çok katmanlı sıkıştırma algoritması, görüntünün kendisine ve önceki sıkıştırmalardan kalan artıklara (residual) art arda uygulanan bir dizi sıkıştırmadan oluşur:

1. Giriş görüntüsü dalgacık (wavelet) tabanında sıkıştırılır. Bu ilk yaklaşım görüntünün genel şeklini korur, yoğunluktaki eğilimi yakalar; deneylerde bu ilk katmanın orijinal görüntünün bir karikatürünü ya da bölütlemesini (segmentasyon) verdiği görülür.
2. Orijinal ile sıkıştırılıp çözülmüş görüntü arasındaki fark hesaplanır. Bu sıkıştırma hatası birinci artıktır. Artıklar dokulardan oluşur.
3. Birinci artık, görüntünün özelliğine göre dalgacık paketleri ya da yerel trigonometrik tabanlarla (yerel kosinüs dönüşümü) sıkıştırılır. Birinci artığın sıkıştırma hatası ikinci artıktır.
4. Artıkta artık bir yapı kalmayana kadar devam edilir.

Öğrenci gösterimi şöyle sadeleştirdi: başlangıç kütüphanesi L_0 (dalgacık) ile görüntü sıkıştırılıp $\hat{R}_0$ elde edilir, kalan R_1 artığıdır. İkinci katmanda R_1 başka bir kütüphaneyle sıkıştırılır ve R_2 kalır. Sıkıştırılmış görüntü, ilk sıkıştırma ile artığın sıkıştırılmış halinin toplamıdır.

Örnek görüntüde: (a) ilk katman, yani dalgacıkla sıkıştırılıp çözülen görüntü; (b) birinci artığın yerel kosinüsle sıkıştırılması (dokuları işleyebilmek için dalgacık yerine bu kullanılır); (c) ikinci artık. İkinci artıkta yalnız gürültü görüldüğü için bir sıkıştırma daha yapmak anlamlı değildir; bu örnekte iki katman kullanılmıştır.

2.3 Bit bütçesinin paylaştırılması

Çalışmada sıkıştırma için önceden bir piksel başına bit (bpp) değeri belirlenir; makale buna bütçe (budget) der. Hoca "bütçe" kelimesinin İngilizcesini sordu; öğrenci "budget" olduğunu ve önceden belirlenen bpp'yi katmanlar arasında nasıl paylaştıracağını ifade ettiğini açıkladı. İki yaklaşım vardır:

- Dalgacık tabanında kolay sıkıştırılabilen düz, pürüzsüz görüntülerde (örneğin Lena) bit bütçesinin büyük kısmı dalgacık katmanına verilir.
- Dokulu görüntülerde (örneğin Roofs; çatı kiremitleri belirgin bir doku oluşturur) bütçenin büyük kısmı doku katmanına verilir.

2.4 Deneyler ve sonuçlar

Deneylerde üç görüntü kullanılmıştır (ekrandan, 00:10:00):

- Roofs, 8 bpp, 512×512 (MIT VisTex veri tabanı): periyodik doku (çatılar) ile düz bölgelerin (cepheler ve gökyüzü) karışımı.
- Barbara, 8 bpp, 512×512: çizgili ve kareli masa örtüsü içeren standart görüntü.
- Clown, 8 bpp, 512×512: palyaçonun standart görüntüsü.

Bu görüntüler geniş düz bölgeler ile uzun salınımlı desenleri birlikte içerdikleri için sıkıştırılması zor görüntülerdir. Karşılaştırma, o zamana kadar en iyi performansı veren Said ve Pearlman'ın SPIHT dalgacık kodlayıcısıyla yapılmıştır. Ölçüt PSNR'dir (tepe sinyal-gürültü oranı, desibel); yüksek olması daha iyidir.

Görüntü	Yöntem	0,0625	0,125	0,25	0,50	0,75	1,00
Roofs	Çok katmanlı	22,61	25,23	28,51	32,79	35,69	38,05
Roofs	SPIHT	21,51	23,77	27,02	31,18	34,18	36,68
Barbara	Çok katmanlı	24,04	26,21	29,18	32,92	35,30	37,27
Barbara	SPIHT	23,35	24,86	27,58	31,39	34,25	36,41
Clown	Çok katmanlı	25,75	28,64	32,34	36,37	38,67	40,18
Clown	SPIHT	25,57	28,23	31,95	35,93	38,27	40,05

(ekrandan, 00:09:35; sütunlar bpp cinsinden oranlardır)

Roofs için 0,125 bpp'de SPIHT 23,77 dB, çok katmanlı yöntem 25,23 dB verir. Görsel karşılaştırmada çatı detayları önceki yöntemde kaybolurken önerilen yöntemde korunmuştur. Barbara'da pantolondaki periyodik doku ve masa örtüsündeki doku daha iyi korunmuş; buna karşılık yüzde bazı bozulmalar görülmüştür. Bunun nedeni başörtüsündeki dokulu yapı ile yüzün pürüzsüz yapısının yan yana olmasıdır; iki yapının birbirine çok yakın olduğu yerlerde yöntem iyi temsil sağlayamamıştır.

Clown görüntüsü literatürde dalgacık dostu bir görüntü olarak bilinir. Bu görüntüde ikinci katmanı kodlamak için dalgacık paketleri yerel kosinüs dönüşümünden daha iyi bir kütüphane olmuştur. PSNR farkı çok küçüktür, ama dalgacık kodlayıcısında gölgeler lekeli görünür ve duvar kağıdının sol üst kısmındaki doku kaybolur; önerilen yöntemde bu doku korunur. Genel olarak çok katmanlı algoritma, PSNR açısından dalgacık kodlayıcısından çok farklı olmasa da görsel kalitede daha iyi sonuç verir.

Hoca sunumun sonunda dalgacık dönüşümünün ayrık kosinüs dönüşümünün (DCT) yerine kullanılan bir yöntem olduğunu, başarısının DCT'den yüksek olduğunu ve SPIHT gibi yöntemlerin arkasında ağaç tabanlı bir ilerleme bulunduğunu belirtti.

> **Ders sonrası not:** Hoca dalgacık dönüşümünün "JPEG'te de kullanıldığını" söyledi. Klasik JPEG DCT kullanır; dalgacık dönüşümü JPEG 2000'de kullanılır. Hoca büyük olasılıkla JPEG 2000'i kastetmiştir.

3. Mobil Uygulamalarda Metin Sıkıştırma: Aritmetik Kodlama, MTF ve BWT

3.1 Çalışmanın amacı ve veri

İkinci sunum, mobil uygulamalarda metin dosyalarının sıkıştırılmasıyla ilgili bir konferans makalesiydi. Mobil cihazlarda kapasite ve bellek büyük bir sorundur; metin dosyası kullanan uygulamalarda belleği fazla harcamamak için düzgün bir sıkıştırma gerekir. Örnek olarak metnini dosyada tutan bir hadis uygulaması alınmıştır.

Veri, Sünen-i İbn Mace adlı eserin (32 kitap) Bahasa çevirisidir; sıkıştırma 1024, 2048, 4096 ve 8192 bayt blok boyutlarında yapılmıştır (ekrandan, 00:20:00).

Soru: Bu metin hangi dilde yazılmış; alfabeler farklıysa resim olarak mı sıkıştırılıyor? **Cevap:** Bahasa çevirisi kullanılmıştır; metin olarak, aritmetik kodlamayla sıkıştırılır.

Üç algoritma, dört yöntem halinde karşılaştırılmıştır:

- Yalnız aritmetik kodlama
- MTF (move-to-front) + aritmetik kodlama
- BWT (Burrows-Wheeler dönüşümü) + aritmetik kodlama
- BWT + MTF + aritmetik kodlama (sırasıyla)

Bu yöntemlerin hepsi, birlikte kullanıldıklarında da, kayıpsızdır.

3.2 Aritmetik kodlama örneği

Makale, “FATIMA AS” dizisini örnek olarak verir. Karakterler alfabetik sıraya dizilir (boşluk dahil), sıklıkları çıkarılır ve bu sıklıklara göre $[0, 1)$ aralığı sırayla bölünür (ekrandan, 00:21:40):

Karakter	Olasılık	Aralık
SPACE	1/10	$0{,}00 \le r < 0{,}10$
A	3/10	$0{,}10 \le r < 0{,}40$
F	1/10	$0{,}40 \le r < 0{,}50$
H	1/10	$0{,}50 \le r < 0{,}60$
I	1/10	$0{,}60 \le r < 0{,}70$
M	1/10	$0{,}70 \le r < 0{,}80$
S	1/10	$0{,}80 \le r < 0{,}90$
T	1/10	$0{,}90 \le r < 1{,}00$

Sonra kodlama algoritmasıyla her karakter için aralık daraltılır. Hoca bunun bildiğimiz normal aritmetik kodlama olduğunu teyit etti.

> **Ders sonrası not:** Slayttaki dizi “FATIMA AS” diye yazılmış, ama tablo 10 karakter, üç A ve bir H içerir; bu olasılıklar “FATIMAH AS” dizisine uyar. Aşağıdaki Örnek Sorular bölümündeki çözüm bu diziyle yapılmıştır.

3.3 Move-to-front kodlama örneği

MTF bir indeksleme yöntemidir. Başlangıçta liste alfabedir. Her karakter için o karakterin listedeki indisi yazılır, sonra karakter listenin en başına alınır ve indeksleme yeniden yapılır. Slayttaki örnekte `aaafhhimst` dizisi kodlanır (ekrandan, 00:22:30):

Okunan	Liste (okunmadan önce, ilk harfler)	İndis
a	abcdefgh…	0
a	abcdefgh…	0
a	abcdefgh…	0
f	abcdefgh…	5
h	fabcdegh…	7
h	hfabcdeg…	0
i	hfabcdegi…	8
m	ihfabcdegjklm…	12
s	mihfabcdegjklnopqrs…	18
t	smihfabcdegjklnopqrt…	19

Çıktı: 0, 0, 0, 5, 7, 0, 8, 12, 18, 19. Tekrar eden karakterler 0 üretir.

3.4 Sıkıştırma oranı ve sonuçlar

Makalede sıkıştırma oranı, sıkıştırılmış dosyanın boyutunun orijinal dosyanın boyutuna bölünüp 100 ile çarpılmasıdır:

$$\text{Oran} = \frac{\text{sıkıştırılmış boyut}}{\text{orijinal boyut}} \times 100$$

Bu tanımda oran ne kadar düşükse sıkıştırma o kadar iyidir. Veri yaklaşık dokuz dosya halinde hazırlanmıştır. Yalnız aritmetik kodlamanın sonuçları (ekrandan, 00:24:15):

Dosya	Boyut (bayt)	Sıkıştırılmış (bayt)
Kitab Hibah.txt	5.937	4.496
Kitab Kafarat.txt	12.934	8.645

Dosya	Boyut (bayt)	Sıkıştırılmış (bayt)
Kitab Hudud.txt	27.851	17.330
Kitab Hukum.txt	59.657	35.983
Kitab Thaharoh.txt	118.991	71.761
Kitab Pelaksanaan	220.133	129.946
Kitab 1-10.txt	618.278	364.932
Kitab 11-32.txt	183.127	107.884
Kitab 1-32.txt	801.405	472.028

En iyi oran en büyük dosyada: $472.028/801.405 \times 100 \approx \%58{,}9$. En kötü oran en küçük dosyada: $4.496/5.937 \times 100 \approx \%75{,}73$.

MTF + aritmetik kodlamada, BWT + aritmetik kodlamada ve üç yöntemin birlikte kullanımında oranlar dört blok boyutu için ayrı ayrı verilmiştir. Öğrencinin aktardığına göre MTF + aritmetik kodlamada en iyi değer en büyük blok boyutunda değil, aradaki bir boyutta yakalanmıştır; öğrenci bunun tekrar eden karakter sayısıyla ilgili olabileceğini söyledi.

Makalenin sonucu: BWT, MTF ve aritmetik kodlama sırayla kullanıldığında en iyi sonuç alınır; BWT büyük blok boyutunda daha iyi sıkıştırır. Makale başta iki soru sorar (bu kombinasyonun Bahasa çevirisi metninde performansı nedir; sıkıştırılmış veri mobil cihazda nasıl kullanılır), ama ikinci soruya pek değinmez.

3.5 Hocanın sorusu: BWT ve MTF ne işe yarar?

Hoca, BWT'yi bu yıl derste anlatmadığını, MTF'yi gösterdiğini; ikisinin aynı kategoride iki yöntem olduğunu belirtti ve sordu.

Soru: BWT ve MTF'yi aritmetik kodlamanın önüne koyduklarında ne yapmaya çalışıyorlar, başarı neden yükseliyor? **Cevap (öğrenci):** BWT kelime içindeki harfleri yan yana koymaya çalışır. **Cevap (hoca):** Görüntü sıkıştırmada görüntüyü önce DCT ya da dalgacık dönüşümünden geçirerek birbirine yakın değerleri yan yana getiriyorduk. Normal bir metinde aynı karakterler arka arkaya gelmez. Daha iyi sıkıştırma için metin, cümle ya da kelime üzerinde işlemler yapılıp aynı karakterler yan yana getirilir. BWT ve MTF, metin üzerindeki dönüşüm yöntemleridir.

> **Hocanın vurgusu:** BWT ve MTF, metinler için dönüşüm yöntemleridir; görüntüde DCT ya da dalgacığın yaptığı işi metinde yaparlar ve entropi kodlayıcının başarısını artırırlar.

4. Transformer ile Kayıpsız Veri Sıkıştırma

4.1 Arka plan

Üçüncü sunum, Transformer kullanımının kayıpsız veri sıkıştırmaya etkisini inceleyen bir konferans sunumuydu. Transformer'lar doğal dil işleme, makine çevirisi ve dil modellemede sık kullanılır. Yapay sinir ağları ve LSTM ile iyi sonuçlar alınsa da sıkıştırma algoritmalarında büyük zaman kayıpları ortaya çıkar; çalışma bu sorunu Transformer ile hafifletmeyi amaçlar.

Kayıpsız sıkıştırma, orijinal verinin hiçbir kayıp olmadan geri elde edilmesini sağlar. Geçmişte istatistiksel sıkıştırma yöntemleri en popüler yöntemlerdi (derste işlenen istatistiksel yöntemler). Daha sonra PAQ tabanlı yöntemler yaygınlaştı: PAQ bir algoritmalar dizisidir, veriyi bit düzeyinde sıkıştırır ve her tür veri için kullanılabilir; farklı alanlar için PAQ1'den PAQ8'e kadar sürümleri vardır. Çok iyi sıkıştırırlar ama çok yavaş ve maliyetlidirler. Bu yüzden sinir ağı tabanlı yöntemler öne çıkmıştır.

PAQ8'deki el yapımı istatistikler, yani önceden hazırlanmış istatistiksel modeller, sinir ağı yöntemlerinde de kullanılır. Bu çalışmada bunun yerine Transformer'ın kendi başına öğrenmesi istenir. Dezavantajı eğitimin yavaş ve maliyetli olmasıdır; çalışma bu maliyeti dengeleyip hızlandırmayı amaçlar.

4.2 Yöntem

Sinir ağıyla kayıpsız sıkıştırmada iki temel unsur vardır: tahminci (predictor) ve entropi kodlama şeması. Çalışmada Transformer-XL modeli geliştirilmiş, deneylerle iyileştirilmiştir. Öğrenciye göre model ayrıntıları makalede çok anlatılmamış, yalnız yapılan değişikliklerin etkisi verilmiştir:

- Katmanlar arası farklı dikkat süreleri incelenmiştir.
- Genelde sinir ağı bir token'ı sıkıştırdıktan sonra üzerinden tekrar geçmez. Bu çalışmada sıkıştırılmış verinin üzerinden tekrar geçilip yeniden eğitime sokulmuştur (revisit). Bu daha verimli bulunmuş, ama daha çok bellek gerektirmiştir.
- Modele girdi olarak n-gram'lar da verilmiştir.

4.3 Deney düzeni ve sonuçlar

Veri: enwik8 (100 MB). Ön işlemede metindeki büyük harfler küçük harfe çevrilmiştir. 12, 18 ve 24 katmanlı; 256, 512 ve 1024 gizli boyutlu Transformer'larla deney yapılmış; 12 katmandan sonra iyileşme görülmediği için 12 katman önerilmiştir. Model ve veri büyüdükçe sonuç iyileşir; yeterli bellek varsa daha büyük modeller önerilir.

enwik8 üzerinde BPC (karakter başına bit) karşılaştırması (ekrandan, 00:37:10):

Yöntem	BPC
gzip -9	2,92
xz -9	1,99
paq8hp12any	1,31
CMIX	1,20
lstm-compress	1,65
Transformer (Bellard 2019)	1,46
LSTM (Bellard 2019)	1,35
Önerilen, tekrar ziyaretsiz	1,33
Önerilen, tekrar ziyaretli	1,20

Slayttaki diğer sonuçlar (ekrandan, 00:40:00):

- Öğrenme oranının etkisi: yakınsama hızı önemli bir parametredir; eğitimin sonuna yaklaştıkça sıkıştırma oranı iyileşir, eğitim sırasında en iyi değer 1,38 BPC olmuştur.
- Büyük-küçük harf ön işlemi 0,01 BPC, n-gram eklemek 0,025 BPC iyileştirme sağlar.
- Daha önce sıkıştırılmış veriyi tekrar ziyaret etmek 1,33 BPC'yi 1,20 BPC'ye düşürür. Modelin geçmiş veriye aşırı uymaması için bırakma (dropout) oranı $p = 0{,}25$ yapıldığında 0,05 BPC iyileşme olur.

Soru (hoca): Bu sempozyum çalışmasında kabaca hangi yönteme vurgu yapılmış? **Cevap:** İstatistiksel yöntemler temel alınarak kendi sinir ağı modellerini geliştirmişler; çalışma bu modelin iyileştirilmesi üzerinedir.

5. Gürültü Giderici Otomatik Kodlayıcıyla Denetimsiz Cümle Sıkıştırma

5.1 Kavramlar

Dördüncü sunum, gürültü giderici otomatik kodlayıcılarla denetimsiz cümle sıkıştırma makalesiydi. Orijinal cümlelere gürültü eklenerek eğitilen bir modelin, dil bilgisi doğruluğunu ve anlamı koruyarak cümleleri ne kadar iyi sıkıştırdığı değerlendirilir.

Soru (hoca): Cümleyi mi sıkıştırıyoruz? **Cevap:** Evet; cümleler sıkıştırılarak metin sıkıştırılır.

- Cümle sıkıştırma (sentence compression): gereksiz bilgiyi kaldırıp orijinal cümlenin dil bilgisini ve önemli içeriğini koruyarak daha kısa bir cümle üretmek.
- Denetimsiz öğrenme: etiketlenmemiş veriden bilinmeyen bir yapıyı tahmin etmek için kullanılan makine öğrenmesi tekniği.

- Gürültü giderici otomatik kodlayıcı (denoising autoencoder): girdisi bilerek bozulmuş verilerle denetimsiz eğitilen sinir ağı; çıktısı orijinal, bozulmamış veridir. Burada cümlelere gürültü eklenerek girdi genişletilir.

Makale, sinirsel metin özetleme tekniklerini, özellikle çıkarımsal (extractive) özetlemeyi cümle sıkıştırmaya uygular. Gürültülü bir girdi dizisinden kelime çıkarma ve yeniden sıralama üzerine eğitilen basit bir gürültü giderici otomatik kodlayıcı, orijinal anlamı koruyan daha kısa diziler üretmeyi öğrenebilir. Modeller hâlâ dil bilgisi ve anlam hatalarına eğilimlidir, ama eşleştirilmiş derlemlere bağımlılığı azaltmak için güçlü bir adım sayılır.

5.2 Eğitim

1. Ekleyerek gürültüleme (additive noising): bir referans cümle alınır; cümle uzatılır ve karıştırılır; model orijinal referans cümleyi kurtarmak üzere eğitilir.
2. Ekleyerek örnekleme (additive sampling): veri setinden rastgele cümleler seçilir, her birinden yerine koymadan birkaç kelime örneklenir ve referans cümleye eklenir.
3. Elde edilen kelime dizisi karıştırılır (shuffle) ve yeni bir girdi elde edilir.
4. Bu girdi gürültü giderici otomatik kodlayıcıya verilir.

Bir kelime grubunu dil bilgisine uygun bir cümleye sıralamanın birçok yolu vardır. Modelin orijinal cümleyi daha iyi kurtarması için, önceden eğitilmiş InferSent modeliyle elde edilen orijinal cümle gömmesi de modele verilir. InferSent, uzun bir öncül metin ve kısa bir hipotez metni verildiğinde öncülün hipotezi gerektirip gerektirmediğini, çeliştiğini ya da nötr olduğunu belirlemek üzere eğitilmiş bir modeldir.

5.3 Veri ve model ayarları

- Veri: haber makaleleri; ilk (uzun) cümleler ve başlıklar (kısa cümleler). Yaklaşık 3,8 milyon eğitim ve 400 bin doğrulama örneği. Metin özetleme görevi için açıklamalı Gigaword kullanılır; eğitimde yalnız referans cümleler kullanılır.
- Tüm modellerde GloVe kelime gömmeleri; sözlük, referanslarda en sık geçen 20 bin kelimeden oluşur.
- Kodlayıcı ve çözücü 512 gizli birimli, 3 katmanlı LSTM'lerdir; batch boyutu 128.

5.4 Değerlendirme

ROUGE puanları (yüksek daha iyi) başka makalelerin sonuçlarıyla karşılaştırılmıştır. F8W, girdinin yalnız ilk 8 kelimesinin alındığı temel yöntemdir. InferSent gömmesi eklenince ROUGE genelde iyileşir. Ablasyon çalışmasında dikkat (attention) kullanımı, bigram karıştırma ve cümle gömmesi eklemenin her birinin performansa olumlu katkı yaptığı görülür (ekrandan, 00:50:00):

Model	R-1	R-2	R-L
1-g shuf (dikkatsiz)	23,01	5,51	20,07
2-g shuf (dikkatsiz)	22,36	5,18	19,60
1-g shuf	27,22	7,63	23,55
2-g shuf	27,72	7,55	23,43
1-g shuf + InferSent	28,12	7,75	24,81
2-g shuf + InferSent	28,42	7,82	24,95

Cümle uzunluğuna göre değerlendirmede girdiler 16-30 ve 31-45 kelime olarak ayrılmış; beklendiği gibi kısa cümlelerde performans daha iyidir. Yazarlar ROUGE'u yeterli görmediği için insan değerlendirmesi de yapmışlardır: ana dili İngilizce olan 6 kişi, 5 modelden rastgele seçilmiş özetleri dil bilgisi ve orijinal anlamı ne kadar koruduğu açısından 1-5 ölçeğinde puanlamıştır. Nicel ve insan değerlendirmesi pek örtüşmemiştir.

Dört örnek cümle gösterilmiştir: modelin bazen orijinal anlamı yakalayan özet ürettiği, bazen düşük ROUGE'a rağmen doğru özetlediği, tamamen karıştırılmış girdiden bile bazı terimleri kurtarabildiği, ama bir kelimenin (örneğin "loss") kaybıyla cümlenin anlamının değiştiği ve

uzun cümlelerde anlamın kaçabildiği görülmüştür. Sonuç: gürültü giderici otomatik kodlayıcılar kötü biçimlenmiş girdilerden bile okunabilir cümle üretmeyi hızla öğrenir; ama dil bilgisi açısından doğru, anlam açısından girdiden farklı çıktılar da üretebilir.

5.5 Hocanın değerlendirmesi

Soru (hoca): Sıkıştırma yaparken cümleyi neden bozuyorlar? Açıldığında cümle orijinal haline dönüyor mu? **Cevap:** Orijinal haline döndürmeye çalışıyorlar; örnek cümlelerde sonuçlara bakıyorlar, ama insan değerlendirmesine göre farklar büyük.

Hoca örneklerin orijinale pek iyi dönmediğini, metinde kayıplı sıkıştırmanın tercih edilmediğini ve yapılmadığını, bu yüzden çalışmanın altında yatan amacı anlayamadığını söyledi.

> **Hocanın vurgusu:** Metin sıkıştırmada kayıplı sıkıştırma tercih edilmez; metnin açıldığında orijinaline dönmesi beklenir.

6. Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma: LSTM ve Transformer

6.1 Temel düzen

Beşinci sunum, Fabrice Bellard'ın sinir ağlarıyla kayıpsız veri sıkıştırma makalesiydi. Kayıpsız sıkıştırma geleneksel tahmin yaklaşımını kullanır: $s_0, \dots, s_{t-1}$ sembolleri bilinirken sinir ağı s_t için bir olasılık vektörü p tahmin eder. s_t'nin gerçek değeri, aritmetik kodlayıcıyla yaklaşık $-\log_2 p(s_t)$ bitle kodlanır. Kod çözücü simetrik olduğu için model parametrelerini iletmeye gerek yoktur; kodlayıcı ve çözücü modelleri aynı şekilde güncelleyerek ilerler.

Ön işleme yoksa s_t, t konumundaki baytı temsil eder ve $N_s = 256$ farklı değer alır. Ön işleme varsa her sembol bir bayt dizisini temsil eder; büyük modellerde yaklaşık 16.000 sembollük sözlük kullanılır. Modellerin yakınsama hızını göstermek için eğitim tek dönemde (epoch) yapılmıştır; örnekler sinir ağından yalnız bir kez geçer. Performans enwik8 ve enwik9 üzerinde değerlendirilir.

6.2 LSTM modeli

LSTM, geri besleme bağlantıları olan bir modeldir. Her hücre, önceki katmandaki karşılık gelen hücrenin çıktısını ve sembolün değerini girdi olarak alır; model L katmandan oluşur. Alex Graves'in "Generating Sequences with Recurrent Neural Networks" modeline benzer bir model kullanılır, ama hücre denklemleri değiştirilir: geleneksel LSTM'den farklı olarak hücre durumu sınırlandırılır. Katman normalizasyonu (layer norm) ile katmanlar normalize edilerek performans iyileştirilir.

Aşırı öğrenme (overfitting), görünmeyen veride genelleme performansını bozan bir sorundur; genelde dengesiz veri kümelerinden ve eğitim verisine çok uyan parametrelerden kaynaklanır, çözümü eğitimde gizli birimleri rastgele bırakan dropout'tur. Bu çalışmada aşırı öğrenme olasılığı çok düşük olduğundan dropout kullanılmaz.

Eğitim (ekrandan, 01:00:00): 20 zaman adımlı ardışık segmentlere düzenli olarak kesilmiş geri yayılım uygulanır; eğitim bölümünün ilk durumları önceki bölümün son değerlerine ayarlanır. Adam optimizasyonu, parametrelerin her birinin öğrenme oranlarının yanında momentum değişikliklerini de önbellekte saklar. Parametre tablosunda girdi gömmeleri hariç parametre sayısı da verilir, çünkü girdi gömmelerinin hesaplama maliyeti ihmal edilebilir; bellek sütunu değerlendirme ve eğitim için ayrılan toplam belleği gösterir. Küçük (small) model hız karşılaştırmalarında kullanılır.

6.3 Transformer modeli

Transformer, girdi verisinin her bölümünün önemini farklı ağırlıklandıran öz-dikkat (self-attention) mekanizmasını benimseyen bir derin öğrenme modelidir. Sıralı girdileri işlemek için tasarlanmıştır ve tüm girdileri bir kerede işler; dikkat mekanizması girdi dizisindeki her konum için bağlam sağlar. Transformer da L katmandan oluşur. Model, "Attentive Language Models Beyond a Fixed-Length Context" çalışmasındaki modele benzer; değişiklik olarak

göreli konum gömmeleri yerine öğrenilen gömmeler kullanılır, çünkü daha basittir ve daha iyi sonuç vermiştir. Aşırı öğrenme olasılığı yine çok düşük olduğundan dropout kullanılmaz.

Eğitimde 64 adımlık ardışık segmentlerle geri yayılım yapılır; 64 ek zaman adımıyla dikkat hesaplaması 128 adım üzerinden yapılır. Adam optimizasyonu kullanılır, 8 dikkat başlığı vardır ve öğrenme oranı doğrusal olarak azalır.

6.4 Kodlayıcı, çözücü ve ön işlemciler

Kodlayıcı ve çözücü aynı model üzerinde simetrik güncelleme yaptığından, her CPU ve işletim sisteminde belirlenimci (deterministic) sonuç vermesi gerekir. Bunun için harici kitaplıklara bağlı olmayan özel bir C kitaplığı (NC kitaplığı) yazılmıştır.

İki ön işlemci kullanılır:

- Küçük LSTM modeli için anlamlı bir karşılaştırma yapmak amacıyla, Byron Knoll'un CMIX çalışmasındaki metin ön işlemcisi ve lstm-compress kullanılır. Yaklaşık 45 bin kelimelik önceden oluşturulmuş bir sözlükteki kelimeler 1-3 baytlık benzersiz kodlarla değiştirilir; büyük harfler kaçış kodlarıyla küçük harfe çevrilir.
- Büyük modeller için alt kelime (subword) tabanlı ön işlemci: ilk geçişte büyük harfler kaçış kodlarıyla küçüğe çevrilir; önünde boşluk olmayan kelimeye boşluk eklenerek kelime biçimlerinin sayısı azaltılır. İkinci geçişte, bayt çifti kodlamasından (byte pair encoding) esinlenerek iyi seçilmiş iki sembol yinelemeli olarak birleştirilip sözlüğe yeni sembol eklenir. Birleştirilecek semboller sıfırıncı derece entropi düşüşünü en büyük yapacak şekilde seçilir; frekansı bir eşiğin altındaki semboller kaldırılır. Kelime ayırıcıları hakkında varsayım yapmadığı için bu geçiş her dilde çalışır. Sözlük yaklaşık 16.000 semboldür.

6.5 Sonuçlar

Sonuçlar bayt ve bpb (girdi baytı başına bit) cinsinden enwik8 (Wikipedia İngilizce sürümünün ilk 100 MB'ı) ve enwik9 (ilk 1 GB) için verilmiştir (ekrandan, 01:05:40):

Program/model (enwik8)	Sıkıştırılmış boyut (bayt)	bpb	Hız (kB/s)
gzip -9	36.445.248	2,92	17.400
xz -9	24.865.244	1,99	1.020
lstm-compress	20.494.577	1,64	4,4
CMIX (v17)	14.877.373	1,19	1,56
LSTM (small)	20.500.039	1,64	41,7
Transformer	18.126.936	1,45	1,79
LSTM (large1)	16.981.765	1,36	3,25
LSTM (large2)	16.791.077	1,34	2,38

Program/model (enwik9)	Sıkıştırılmış boyut (bayt)	bpb
gzip -9	322.591.995	2,58
xz -9	197.331.816	1,58
CMIX (v17)	116.394.271	0,93
Transformer	130.440.948	1,04
LSTM (large1)	128.292.351	1,03
LSTM (large2)	125.623.896	1,00

En iyi oran her iki veri setinde CMIX'e aittir (enwik8'de 1,19, enwik9'da 0,93 bpb). Önerilen modeller son teknoloji dil modelleriyle doğrudan karşılaştırılamaz: veri üzerinden tek geçiş yapılır, BPB son 5 MB yerine tüm veri üzerinde hesaplanır ve ön işleme sözlüğünün boyutu hesaba katılmaz. LSTM ve Transformer'da açma hızı sıkıştırma hızından düşüktür; buna rağmen küçük LSTM modeli benzer bir model olan lstm-compress'ten çok daha hızlıdır. Hız, daha iyi optimize edilmiş matris çarpımı ya da daha büyük batch boyutuyla artırılabilir. CMIX'in performansına ulaşılamamış, ama bu kadar basit bir modelle iyi sonuç alınmıştır.

Ders sonrası not: Sunumda CMIX'in "1,56 ile sıkıştırma hızı en hızlı model" olduğu söylendi. Tabloda 1,56 kB/s sıkıştırma hızıdır ve tablodaki en düşük hızdır; CMIX en iyi oranı veren ama en yavaş programdır. En hızlı program gzip -9'dur (17.400 kB/s).

Dosya ofsetine göre sıkıştırma oranının değişimi grafiğinde (kırmızı Transformer, mavi LSTM), aynı parametre aralığında Transformer'ın LSTM'den daha yavaş yakınsadığı görülür; Transformer'ı iyileştirmek için daha çok deney gerekir. Sonuç olarak LSTM tabanlı kayıpsız sıkıştırmanın pratik bir uygulaması sunulmuştur: LSTM yavaştır ama kodu basittir, bellek tüketimi benzer oran veren sıkıştırıcılara göre makuldür ve özel donanımdan büyük ölçüde yararlanabilir. Daha büyük modeller, daha iyi gradyan optimize edici, hiperparametre ayarı ve daha iyi ön işlemciyle oran daha da iyileştirilebilir.

7. DNN ile Sismik Veri Sıkıştırma

7.1 Sismik veri ve klasik yöntemler

Altıncı sunum, derin sinir ağı (DNN) kullanarak sismik veri sıkıştırma makalesiydi. Hoca "DNN"in ne olduğunu sordu; öğrenci bunun derin sinir ağı (deep neural network) kısaltması olduğunu açıkladı.

Sismik veri, yer altı yapısı hakkında bilgi almak için toplanır (petrol ve gaz araştırmaları). Belli aralıklarla sensörler yerleştirilir, sinyaller veri işleme merkezine gönderilir. Veri çok büyük olduğu için iletim yüksek bant genişliği gerektirir; bu yüzden sıkıştırma kullanılır.

Kayıpsız yöntemler yüksek sıkıştırma performansı göstermez; kayıplı sıkıştırmada da olduğu gibi dönüşüm ve tahmine dayalı yaklaşımlar kullanırlar ve tamsayı değerler için 5:1, float değerler için 2:1 sıkıştırma sağlarlar (ekrandan, 01:20:00). Daha yüksek performans için kayıplı sıkıştırma kullanılır ve kayıp en aza indirilmeye çalışılır. Her kayıplı sıkıştırma algoritmasında üç adım vardır:

- Dekorelasyon: sinyalin otokorelasyonunu ve diğer özelliklerini koruyarak çapraz korelasyonu azaltır. Dalgacıklar, yerel trigonometrik fonksiyonlar ya da ikisinin kombinasyonu kullanılır; bu çalışmada dalgacıklar kullanılmıştır.
- Kuantalama.
- Kodlama: önceki adımdan kalan artıklar ve katsayılar sabit uzunluklu (fixed-length) ya da değişken uzunluklu (variable-length) kodlamayla kodlanır.

7.2 Tahmine dayalı yöntem

Ana fikir şudur: tüm veriyi göndermek yerine bir sinir ağı eğitilir. Sinir ağının çıktısının varyansı, verinin varyansından her zaman düşük olacağı için sıkıştırma kolaylaşır. DNN, $n-1$. adıma kadar olan bilgiyi kullanarak yinelemeli olarak n. adımı tahmin eder.

Mimari (ekrandan, 01:21:50):

```
(a) Gönderici:
  Girdi verisi -> [Tahminci eğitimi -> Artık kodlama] -> Kodlanmış artık
                                    |
                                    +--> Tahminci kurulumu ve veri segmentleri

(b) Alıcı:
  Tahminci kurulumu ve veri segmentleri -> Tahminci --+
  Kodlanmış artık ---------------------------------> (+) -> Yeniden kurulmuş veri
                                 (çıktı tahminciye geri beslenir)
```

- Örneklerin bir kısmı gönderici taraftaki modeli eğitmek için kullanılır; bu veriler doğrudan alıcıya gönderilir.
- Model ağırlıkları ve katsayılar alıcıya gönderilir; iki tarafta da aynı mimari kurulur.
- Artıklar K-Means algoritmasıyla kuantalanır; kuantalanan değerler Huffman ile kodlanıp alıcıya gönderilir.

Alıcı tarafta iki tahmin yöntemi karşılaştırılmıştır. LPC (doğrusal tahmin) ortalama kare uydurma ile regresyon yapar; ama doğrusal modeller karmaşık ilişkileri iyi modelleyemez. Başarıyı artırmak için her iz (trace, art arda gönderilen her kayıt) kendi içinde normalize edilmiştir.

DNN tarafında her zaman modelin önceki çıktısı ve artık değeri bir sonraki tahmin için girdi olarak verilir. Gizli katman sayısı arttıkça kaybolan gradyan (vanishing gradient) sorunuyla karşılaşılmıştır.

Hoca şemaya bakarak modelin, girdiden H tane gizli birime tam bağlı, düz çok katmanlı bir yapay sinir ağı olduğunu belirtti; öğrenci de makalede mimarinin ayrıntılı anlatılmadığını, girdilerin ağırlıklarla çarpılıp gizli katmana gönderildiği düz bir çok katmanlı ağ olduğunu söyledi. Artık kodlamada sabit uzunluklu kodlama ve Huffman ayrı ayrı denenmiştir.

7.3 Deneyler

Deneyler East Texas veri seti üzerinde yapılmıştır; veri seti açık kaynak olarak vardır ama model açık kaynak değildir. Her iz gönderimden önce 0-1 aralığına normalize edilmiştir. Veri eğitim, doğrulama ve test için üç eşit parçaya bölünmüş; yeniden yapılandırma ölçütü olarak SNR (sinyal-gürültü oranı) kullanılmıştır. SNR'nin yüksek olması sinyaldeki gürültü oranının düşük olduğunu, yani daha başarılı bir sıkıştırmayı gösterir.

LPC ve DNN için SNR (dB) (ekrandan, 01:24:10); N veri segmenti (bir gönderimdeki veri sayısı), L_Q kuantalama seviyesi:

N	LPC 16	LPC 32	LPC 64	DNN 16	DNN 32	DNN 64
5	23,1	28,96	30,27	43,02	52,15	58,31
10	26,56	27,64	29,55	42,07	51,45	57,41
15	25,86	28,69	29,88	42,81	52,81	58,82

Örneğin $N = 5$, $L_Q = 16$ için SNR 23,1 dB'den 43,02 dB'ye çıkar; değerler neredeyse iki katına çıkmıştır. Kuantalama seviyesi arttıkça yeniden yapılandırma başarısı artar. Orijinal ve yeniden kurulmuş veri arasındaki fark çok küçüktür.

Huffman kodlaması, sabit uzunluklu kodlamaya göre sıkıştırma oranını artırmıştır; daha sık görülen sinyal genlikleri daha az bitle ifade edildiği için ortalama kod uzunluğu kısalır. Sonuç olarak gerçek veri üzerinde DNN, LPC'den çok daha başarılıdır; karmaşık ilişkili verilerde sinir ağları ilişkileri daha iyi modeller. Yüksek bozulma olmadan, 40 dB'nin üzerinde SNR ile yaklaşık 10:1 sıkıştırma elde edilmiştir.

> **Ders sonrası not:** Öğrenci sunumun sonunda karşılaştırma için "tamsayı değerler için 2:1, float değerler için 5:1" dedi. Slayttaki doğru değerler tamsayı için 5:1, float için 2:1'dir (ekrandan, 01:20:00).

Hoca şemayı tekrar açtırıp akışı özetledi: girdi sismik veridir; tahminci eğitimi ve artık kodlama kutusunda kuantalama ve kodlama yapılır; kodlanmış artık ile tahminci kurulumu ve veri segmentleri gönderilir, alıcı bunlarla veriyi açar.

8. Evrişimsel RNN ile Göğüs Röntgeni Sıkıştırma

8.1 Motivasyon ve veri

Yedinci sunum, röntgen görüntülerinin evrişimsel yinelemeli sinir ağlarıyla (convolutional RNN) sıkıştırılması makalesiydi. Tıbbi görüntülerin verisi çok büyüktür; yeterli sağlık çalışanı olmadığından görüntülerin yorumlanmak üzere uzmana gönderilmesi ve bunun için sıkıştırılması gerekir. Araştırma sorusu, belli bir veri setinde genel yöntemlerden daha iyi sıkıştırma yapılıp yapılamayacağıdır.

Soru (hoca): Kayıpsız sıkıştırma mı yapıyorlar? **Cevap:** Hayır, kayıplı.

Hoca kayıplı sıkıştırmadan her zaman korktuğunu, çünkü röntgendeki küçücük bir lezyonun sıkıştırmada yok olabileceğini ve doktorun onu göremeyebileceğini söyledi; öğrenci makalenin de bunu araştırdığını belirtti.

> **Hocanın vurgusu:** Tıbbi görüntülerde kayıplı sıkıştırma tehlikelidir; küçük bir lezyon kaybolabilir.

Veri, ChestX-ray8 veri setidir; 8 sınıf vardır ve her biri farklı bir hastalığı temsil eder. 1024×1024 görüntülerden 128×128 ve 256×256'lık kesitler (crop) alınmış, iki boyut birbiriyle karşılaştırılmıştır. Eğitim için 20.000, doğrulama için 1.000, test için 500 örnek ayrılmıştır; hastalık ve kişi özelliklerinden bağımsız olunması için kesitler karıştırılarak rastgele seçilmiştir (ekrandan, 01:30:00). Hastalığın bulunduğu bölgeler veri setinin kendisinde işaretlenmiştir.

8.2 Model

Model şeması (ekrandan, 01:32:50) üstte kodlayıcıyı (sıkıştırma alt ağı), altta çözücüyü (açma alt ağı) gösterir:

```
Kodlayıcı:
  girdi --+--> Pool 2x2 --> Conv 1x1x64, adım 1x1 --+
          |                                         (ortalama) --> RNN-Conv 3x3x256, adım 2x2
          +--> Conv 3x3x64, adım 2x2 ---------------+
        --> RNN-Conv 3x3x512, adım 2x2 --> RNN-Conv 3x3x512, adım 2x2
        --> Darboğaz (bottleneck) 1x1xβ, adım 1x1 --> H/16 x W/16 x β

Çözücü:
  H/16 x W/16 x β --> Conv 1x1x512
    --> [RNN-Conv 2x2x512 ve RNN-Conv 1x1x512, ortalama] --> D2S
    --> [RNN-Conv 3x3x512 ve RNN-Conv 1x1x512, ortalama] --> D2S
    --> [RNN-Conv 2x2x256 ve RNN-Conv 1x1x256, ortalama] --> D2S
    --> [RNN-Conv 2x2x128 ve RNN-Conv 1x1x128, ortalama] --> D2S
    --> Conv 1x1x1 --> çıktı
```

- Bu çalışmayı diğerlerinden ayıran nokta basit özelliklere önem vermesidir: ilk işlem iki kolda yapılır. Bir kolda havuzlama (pooling) ve evrişim, diğerinde adımlı (stride) evrişim uygulanır; iki kolun çıktılarının ortalaması alınır.
- RNN-Conv katmanı başka bir makalede önerilmiş bir yapıdır: LSTM'de girdiler ağırlıklarla çarpılırken burada girdiye ve gizli duruma evrişim uygulanır. Kapılar dört parçaya bölünüp birlikte hesaplanır, aktivasyon fonksiyonları uygulanır, hücre ve gizli durumlar oluşturulup bir sonraki adıma verilir.
- 128×128 girdi her adımda yarıya iner: 64×64, 32×32, 16×16 ve sonunda 8×8. Yani her kenar 1/16'ya düşer.
- Darboğazdaki β parametresi sıkıştırma oranını ayarlar. Sunumda $\beta = 256/R$ formülü verildi; R sıkıştırma oranıdır. $R = 2$ için $\beta = 256/2 = 128$ olur.
- Çözücüde depth-to-space (D2S) bir yeniden şekillendirmedir: derinlikten alıp yüzey alanını genişletir. 8×8'lik tensör D2S'den sonra 16×16 olur, buna karşılık derinlik 4'te 1'ine düşer (512 kanaldan 128'e). Sonunda yine 128×128'lik bir görüntü elde edilir.

8.3 Metrikler ve sonuçlar

- PSNR (tepe sinyal-gürültü oranı): ortalama kare hata (MSE) paydada olduğu için büyük olması daha iyidir.
- SSIM (yapısal benzerlik indeksi): yine büyük olması daha iyidir.

128×128 kesitler için sonuçlar (ekrandan, 01:34:15):

Oran	Yöntem	SSIM	PSNR (dB)
2	Toderici ve ark.	0,9387	32,1130
2	JPEG 2000	0,9801	25,1276
2	Önerilen	0,9645	36,0152
4	Toderici ve ark.	0,9119	30,8696
4	JPEG 2000	0,9681	25,0529
4	Önerilen	0,9592	35,9795
8	Toderici ve ark.	0,8970	30,4720
8	JPEG 2000	0,9409	24,8608
8	Önerilen	0,9579	35,9325

Öğrenciye göre Toderici ve arkadaşlarının çalışması, daha önce görülen ve bu çalışmanın temel aldığı önceki çalışmadır. Öğrenci, oran 2'de JPEG 2000'in daha iyi sonuç verdiğini, oran arttıkça önerilen yöntemin JPEG 2000'i geçtiğini söyledi; tabloda bu SSIM için geçerlidir, PSNR'de önerilen yöntem her oranda en iyisidir. Oran 8'de önerilen yöntem hem SSIM'de hem PSNR'de en iyisidir. Makalenin iddiası, belirli bir veri setinde derin öğrenmeyle genel yöntemlerden daha iyi sıkıştırma yapılabileceğidir. 256×256 görüntülerde sonuçlar oranla neredeyse değişmemiştir. Görsel olarak önceki çalışma bulanık sonuç verirken önerilen yöntem daha net görüntü verir.

9. Burrows-Wheeler Dönüşümü

9.1 Tanım ve evreler

Sekizinci sunumda bir öğrenci Burrows-Wheeler dönüşümünü (BWT) anlattı. Hoca, BWT'yi bu yıl derste anlatmadığını, seminer konularında bulunduğu için öğrenciye verdiğini söyledi. Öğrenciye göre Burrows ve Wheeler'ın çalışması 1994'te Kaliforniya'da geliştirilmiştir; algoritma metin sıkıştırma için tasarlanmıştır ve birçok yerde kullanılır.

BWT, karakter dizisini, aynı karakterler arka arkaya gelecek şekilde yeniden düzenler. En önemli özelliği, dönüşümü geri almak için orijinal karakterlerin dışında ek bir veri saklamaya gerek olmamasıdır; yalnız orijinal dizinin hangi satırda olduğu bilinmelidir.

Evreler (ekrandan, 01:40:00):

```
Sıkıştırma: bloklara ayrılmış veri -> BWT -> MTF -> Huffman veya aritmetik kodlama -> sıkıştır
Açma:       sıkıştırılmış veri blokları -> ters Huffman veya ters aritmetik -> ters MTF -> ter
```

9.2 abracadabra örneği

Slayttaki örnek "abracadabra" kelimesidir (ekrandan, 01:40:45). (a) sütununda kelimenin döndürmeleri yazılır: her adımda ilk karakter sona alınır.

```
(a) döndürmeler      (b) sıralanmış
abracadabra          bracadabraa
bracadabraa          dabraabraca
racadabraab          braabracada
acadabraabr          cadabraabra
cadabraabra     ->   abracadabra   (5. satır: orijinal)
adabraabrac          racadabraab
dabraabraca          raabracadab
abraabracad          adabraabrac
braabracada          abraabracad
raabracadab          acadabraabr
aabracadabr          aabracadabr
```

(b) sütununda döndürmeler sıralanmış ve ilk sütun kutu içine alınmıştır. Bu sıralamada her satır, ilk karakterinden önce gelen bağlama (satır sağdan sola okunduğunda) göre dizilmiştir. Çıktı, (b)'nin ilk sütunudur: `bdbcarraaaa`. Orijinal kelime 5. satırdadır; bu indis de gönderilir.

9.3 Ters dönüşüm

Ters dönüşüm tablosu (ekrandan, 01:42:30):

Sıra	Girdi dizisi	Sıralanmış girdi	SıraBul çıktısı	Çıktı dizisi
1	b1	a1	6	a1
2	d1	a2	9	b1
3	b2	a3	7	r1
4	c1	a4	8	a4
5	a1	a5	1	c1
6	r1	b1	10	a2

Sıra	Girdi dizisi	Sıralanmış girdi	SıraBul çıktısı	Çıktı dizisi
7	r2	b2	11	d1
8	a2	c1	2	a3
9	a3	d1	3	b2
10	a4	r1	4	r2
11	a5	r2	5	a5

Adımlar:

1. Girdi dizisi (`bdbcarraaaa`), aynı karakterler numaralanarak yazılır (a1, a2, …).
2. Girdi sıralanır.
3. SıraBul çıktısı: girdi dizisinin her elemanının sıralanmış dizideki yeri. Örneğin girdideki b1 sıralanmış dizide 6. sıradadır, d1 9. sıradadır, a1 1. sıradadır.
4. Gönderilen indisten (5) başlanır: girdi dizisinin 5. elemanı a1 yazılır; SıraBul'un 5. değeri 1'dir, girdi dizisinin 1. elemanı b1 yazılır; SıraBul'un 1. değeri 6'dır, 6. eleman r1 yazılır; böyle devam edilir.

Sonuçta çıktı dizisi a1, b1, r1, a4, c1, a2, d1, a3, b2, r2, a5, yani "abracadabra" olur (ara adımlar ve doğrulama python ile yapıldı).

Hoca sınıfın bu kısmı anlamadığını düşünerek öğrenciden (b) kısmını tekrar anlatmasını istedi. Öğrenci, (a)'da her satırdaki ikinci karakterin bir sonraki satırın birinci karakteri olduğunu, ters dönüşümde ise SıraBul çıktısının karışık (6, 9, 7, …) geldiğini, alınacak dizinin ilk karakterinin 5. satırdan başladığını açıkladı.

9.4 Kullanım ve hocanın yorumu

Sıkıştırmada veri bloklara ayrılır ve her bloğa BWT uygulanır. bzip2, BWT ile Huffman kodlamasını kullanır ve geleneksel yöntemlere göre daha etkindir.

Hoca BWT'nin bakınca çok basit göründüğünü, ama bu kadar basit bir şeyi bile bulamadığımızı düşünmemiz gerektiğini söyledi.

Soru (hoca): Sana bir kelime versem BWT'den rahatlıkla geçirebilir misin? **Cevap:** Evet, mantığı anlaşıldıysa geçirilebilir.

10. Proje Gösterimleri: Huffman ile Görüntü Sıkıştırma

Hoca projelerin final gününde değerlendirileceğini, ama önceden bitirenlerin gösterebileceğini söyledi; bu derste iki öğrenci Huffman projesini gösterdi.

10.1 Birinci proje: BMP görüntülerde Huffman

Uygulama bir görüntüyü sıkıştırır, sonra açar; sıkıştırılmış görüntünün kod tablosunu ve Huffman ağacını gösterir. Kod tablosunda en sık kullanılan sembole en kısa, en az kullanılana en uzun kod verilir. Üç BMP görüntüyle denenmiştir (ekrandan, 01:50:00):

- Birinci görüntü: 2.359.326 bayt → 2.265.942 bayt; kazanç %4. Hesap: $1 - 2.265.942/2.359.326 \approx 0{,}0396$.
- İkinci görüntü: 735 KB → 593 KB; kazanç %19 ($1 - 593/735 \approx 0{,}193$).
- Üçüncü görüntü: 1.130.022 bayt → 782.515 bayt; kazanç %31 ($1 - 782.515/1.130.022 \approx 0{,}308$).

Bu projede "sıkıştırma oranı", boyuttaki azalma yüzdesi olarak verilmiştir (Bölüm 3.4'teki tanımın tersi yönde). En kısa kod 5 bit uzunluğundaydı; örneğin 210 değerli piksel sembolü en yoğun kullanılan semboller arasındadır.

Soru (hoca): Huffman'dan önce bir kuantalama yapıp renk sayısını azaltsaydın başarı yükselir miydi? **Cevap:** Renk sayısı azalsaydı kesinlikle yükselirdi; proje orijinal görüntü üzerinden yapılmıştır.

Soru (hoca): Açan tarafın kodlara ihtiyacı var; header'da ağacı mı gönderdin, kodları mı? **Cevap:** Önce her sembolün frekans bilgisiyle bir alfabe oluşturuluyor, sonra kodlar gönderiliyor; daha sonra öğrenci ağaç bilgisinin header'a konduğunu belirtti. Hoca, header'ın bu durumda biraz şiştiğini söyledi.

Hoca, %31 kazanç alınan görüntüde çok sınırlı renk (siyah, beyaz, birkaç mavi tonu) göründüğünü, ama gözün ayırt edemediği çok sayıda ton bulunduğunu, bunlar kuantalama ile azaltılsa sıkıştırmanın çok daha başarılı olacağını söyledi. Öğrenci, %4 kazanç alınan görüntüyü önceden bir araçla %50 küçülttüğünü belirtti; hoca kazancın düşüklüğünün bununla ilgili olduğunu söyledi. Hoca ayrıca daha az renkli bir görüntü seçilirse ağacın küçüleceğini önerdi.

Kodun akışı: piksel değerleri okunur; her sembolün frekansı ölçülüp tablo oluşturulur; tablo sıralanır; Huffman ağacı kurulur; açarken aynı ağaç üzerinden geri dönülür.

> **Hocanın vurgusu:** Gözün ayırt edemediği renkler kuantalama ile azaltılırsa Huffman ağacı küçülür, header küçülür ve sıkıştırma başarısı artar.

10.2 İkinci proje: dosya ve bitmap sıkıştırma

İkinci öğrenci önce internetten bulduğu dört hazır Huffman kodunu derleyip inceledi, sonra kendi Huffman altyapısını yazdı. Çalışma üç kısımdan oluşur:

- Genel dosya sıkıştırma: herhangi bir dosya için başlık (header) oluşturulur: sembol ve tekrar sayısı. Kodlayıcı (`WriteHeader` ile) header'ı ve veriyi yazar, çözücü okur. Hoca header'da görüntü boyutunun, sembol sayısının ve tablonun bulunduğunu teyit etti.
- Ağacın adım adım gösterimi: parametrik bir ayarla ağacın her birleştirme adımı ayrı ayrı gösterilir.
- Görüntü sıkıştırma: 256 renk paletli bitmap üzerinde çalışılır. Dosya ikiye bölünür: başlık (ve renk tablosu) kısmına dokunulmaz; raster (piksel) verisi önceki mekanizmayla Huffman ile kodlanır ve kendi başlığı oluşur.

Basit örnek: 16×4 = 64 piksellik bir bitmap; görüntüde mor, kırmızı, gri, siyah, yeşil ve beyaz bölgeler vardır (ekrandan, 02:00:00). Ağaç adım adım kurulur, son adımda bütün ağaç oluşur.

Daha büyük bir örnek görüntünün rapor dosyasında (ekrandan, 02:02:30):

- Entropi: 3,013041 bit/sembol
- Ortalama kod uzunluğu: 3,03688 bit/sembol
- Orijinal: 133.136 bayt, sıkıştırılmış: 50.540 bayt (yalnız veri kısmı, header hariç)
- Beyaz (255) sembolü 70.012 kez geçer ve tek bitlik 1 kodunu alır.

Ortalama kod uzunluğu entropiye çok yakındır ve olması gerektiği gibi entropiden biraz büyüktür. Bu görüntü avantajlıdır, çünkü beyaz çok fazladır. Lena görüntüsünde (256 renk) histogram çok daha düzgün dağılmıştır; entropi 8 bite yakın olduğundan sıkıştırma çok azdır: yaklaşık 262 binden 258 bin bayta. Program son adımda orijinal dosyayla açılmış dosyayı bayt bayt karşılaştırır (compare file). Bitlerin yazılıp okunması için bit paketleme (bit packing) yapılır: bir baytlık tampon dolduğunda (8 bit) dosyaya yazılır.

Soru (hoca): Sıkıştırılmış 50 KB'lık dosyanın başında header bilgisi de var mı? **Cevap:** Hayır, yalnız veri kısmıdır.

Örnek Sorular

Soru 1 (ekrandan, 00:22:30): Başlangıç listesi alfabe (a-z) olan move-to-front kodlamasıyla `aaafhhimst` dizisini kodlayın.

> **Çözüm:** a → 0, a → 0, a → 0, f → 5 (liste: f a b c d e g …), h → 7 (h f a b c d e g …), h → 0, i → 8, m → 12, s → 18, t → 19. Çıktı: 0, 0, 0, 5, 7, 0, 8, 12, 18, 19.

Soru 2 (ekrandan, 00:21:40): Tablodaki olasılıklarla (SPACE 1/10, A 3/10, F, H, I, M, S, T her biri 1/10; aralıklar alfabetik sırayla) "FATIMAH AS" dizisini aritmetik kodlamayla kodlayın; son aralık nedir?

> **Çözüm (derste verilmedi):** Her karakterde alt ← alt $+ w \cdot a$, üst ← $\text{alt}_{eski} + w \cdot b$ (w = aralık genişliği, $[a, b)$ karakterin aralığı). F: $[0{,}4;\ 0{,}5)$; A: $[0{,}41;\ 0{,}44)$; T:

$[0{,}437;\ 0{,}44)$; I: $[0{,}4388;\ 0{,}4391)$; M: $[0{,}43901;\ 0{,}43904)$; A: $[0{,}439013;\ 0{,}439022)$; H: $[0{,}4390175;\ 0{,}4390184)$; boşluk: $[0{,}4390175;\ 0{,}43901759)$; A: $[0{,}439017509;\ 0{,}439017536)$; S: $[0{,}4390175306;\ 0{,}4390175333)$. Bu aralıktaki herhangi bir sayı diziyi temsil eder. Son aralığın genişliği $(3/10)^3 \cdot (1/10)^7 = 2{,}7 \cdot 10^{-9}$'dur (python ile doğrulandı).

Soru 3 (ekrandan, 00:24:15): Yalnız aritmetik kodlamada en büyük dosya 801.405 bayttan 472.028 bayta, en küçük dosya 5.937 bayttan 4.496 bayta inmiştir. Oran = sıkıştırılmış / orijinal × 100 tanımıyla iki oranı hesaplayın.

> **Çözüm:** $472.028/801.405 \times 100 \approx \%58{,}9$ (en iyi); $4.496/5.937 \times 100 \approx \%75{,}73$ (en kötü). Bu tanımda düşük oran daha iyi sıkıştırma demektir.

Soru 4 (derste sözlü, 00:27:10): BWT ve MTF aritmetik kodlamanın önüne konunca başarı neden artar?

> **Çözüm:** Normal metinde aynı karakterler arka arkaya gelmez. BWT aynı karakterleri yan yana getirir, MTF de tekrarları küçük sayılara (çoğunlukla 0) çevirir. Görüntüde DCT ya da dalgacık dönüşümünün yaptığı gibi, bu metin dönüşümleri entropi kodlayıcının daha iyi sıkıştırmasını sağlar.

Soru 5 (ekrandan, 01:40:45 ve 01:42:30): "abracadabra" kelimesinin BWT çıktısı `bdbcarraaaa`, orijinal satır indisi 5'tir. Ters dönüşümle kelimeyi geri elde edin.

> **Çözüm:** Girdi: b1 d1 b2 c1 a1 r1 r2 a2 a3 a4 a5. Sıralanmış: a1 a2 a3 a4 a5 b1 b2 c1 d1 r1 r2. SıraBul (girdinin her elemanının sıralanmış dizideki yeri): 6, 9, 7, 8, 1, 10, 11, 2, 3, 4, 5. 5. konumdan başla: girdi[5] = a, SıraBul[5] = 1 → girdi[1] = b, SıraBul[1] = 6 → r, SıraBul[6] = 10 → a, SıraBul[10] = 4 → c, SıraBul[4] = 8 → a, SıraBul[8] = 2 → d, SıraBul[2] = 9 → a, SıraBul[9] = 3 → b, SıraBul[3] = 7 → r, SıraBul[7] = 11 → a. Sonuç: abracadabra.

Soru 6 (derste sözlü, 01:48:42): Huffman ile görüntü sıkıştırmadan önce kuantalama yapıp renk sayısını azaltmak başarıyı nasıl etkiler?

> **Çözüm:** Farklı sembol sayısı azalır, olasılıklar daha az sayıda sembolde toplanır; Huffman ağacı ve header küçülür, ortalama kod uzunluğu düşer. Sıkıştırma başarısı artar; ama bu artık kayıplı bir sıkıştırmadır.

Soru 7 (derste sözlü, 01:50:13): Huffman ile sıkıştırılmış dosyayı açan taraf kodları nasıl bilir?

> **Çözüm:** Kodlayıcı, ağacı (ya da sembol frekanslarını veya kod tablosunu) header'a koyar; çözücü aynı ağacı yeniden kurar. Bu bilgi header'ı büyütür ve net kazancı azaltır.

Soru 8 (ekrandan, 02:02:30): Bir görüntüde 133.136 sembol (bayt) vardır; Huffman ile ortalama kod uzunluğu 3,03688 bit, entropi 3,013041 bittir. Sıkıştırılmış veri boyutu yaklaşık kaç bayttır? Ortalama kod uzunluğu ile entropi arasındaki ilişki nedir?

> **Çözüm (derste verilmedi):** $133.136 \times 3{,}03688/8 \approx 50.540$ bayt; rapordaki 50.540 bayt ile uyuşur (python ile doğrulandı). Huffman kodunun ortalama uzunluğu entropiden küçük olamaz ve entropi + 1'den küçüktür: $3{,}013 \leq 3{,}037 < 4{,}013$.

Dersin Toparlaması

Bu derste sekiz seminer sunumu ve iki proje gösterimi yapıldı. Klasik tarafta, çok katmanlı görüntü temsili dalgacık ve yerel kosinüs dönüşümlerini artıklar üzerinde art arda kullanarak dokulu görüntülerde SPIHT'ten daha iyi görsel kalite verdi; metin tarafında BWT, MTF ve aritmetik kodlamanın sırayla kullanılmasının en iyi sonucu verdiği görüldü. Hoca, BWT ve MTF'nin metinde, görüntüdeki DCT ve dalgacık gibi bir dönüşüm görevi gördüğünü vurguladı; BWT'nin ileri ve ters dönüşümü abracadabra örneğiyle adım adım işlendi.

Sinir ağı tarafında ortak fikir, bir modelin sonraki sembolün olasılığını tahmin etmesi ve aritmetik kodlayıcının bu olasılıkla kodlamasıdır: Transformer-XL tabanlı model ve Bellard'ın LSTM ve Transformer modelleri enwik8'de CMIX'e yaklaşan BPC değerleri verdi. Kayıplı tarafta sismik veride DNN tahmini, K-Means kuantalama ve Huffman ile LPC'ye göre yaklaşık iki kat SNR; röntgen görüntülerinde evrişimsel RNN ile JPEG 2000'den yüksek PSNR elde edildi. Hoca kayıplı metin sıkıştırmanın tercih edilmediğini ve tıbbi görüntüde kayıplı sıkıştırmanın

riskli olduğunu vurguladı. Proje gösterimlerinde Huffman kazancının görüntünün renk dağılımına bağlı olduğu, entropisi 8 bite yakın görüntülerin az sıkıştığı ve renk sayısını kuantalamayla azaltmanın başarıyı artıracağı tartışıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.